

# React Query Guide

React Query (now TanStack Query) is a powerful data-fetching library for React that helps manage server state, caching, and synchronization with ease. It simplifies data fetching and updating processes while improving performance and developer experience.

## useQuery

`useQuery` is used for fetching data. It caches the result and automatically refetches when dependencies change.

```
import { useQuery } from '@tanstack/react-query';

function Posts() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['posts'],
    queryFn: fetchPosts,
  });

  if (isLoading) return <p>Loading...</p>;
  if (error) return <p>Error: {error.message}</p>;

  return (
    <ul>
      {data.map(post => <li key={post.id}>{post.title}</li>)}
    </ul>
  );
}
```

## useMutation

`useMutation` is used for creating, updating, or deleting data. Unlike `useQuery`, it does not run automatically and must be triggered manually.

```
import { useMutation, useQueryClient } from '@tanstack/react-query';

function AddPost() {
  const queryClient = useQueryClient();
  const mutation = useMutation({
    mutationFn: addPost,
    onSuccess: () => {
      queryClient.invalidateQueries(['posts']);
    }
  });

  return (
    <button
      onClick={() =>
        mutation.mutate({ title: 'New Post', content: 'Hello world!' })
      }
    >
      Add Post
    </button>
  );
}
```

## useInfiniteQuery

`useInfiniteQuery` helps in fetching paginated data. It provides utilities like `fetchNextPage` and `hasNextPage`.

```
import { useInfiniteQuery } from '@tanstack/react-query';

function InfinitePosts() {
  const { data, fetchNextPage, hasNextPage, isFetchingNextPage } =
    useInfiniteQuery({
      queryKey: ['posts'],
      queryFn: ({ pageParam = 1 }) => fetchPosts(pageParam),
      getNextPageParam: (lastPage, allPages) => lastPage.nextPage,
    });

  return (
    <div>
      {data?.pages.flatMap(page =>
        page.items.map(post => <p key={post.id}>{post.title}</p>)
      )}
      <button onClick={() => fetchNextPage()} disabled={!hasNextPage}>
        {isFetchingNextPage ? 'Loading...' : 'Load More'}
      </button>
    </div>
  );
}
```

## React Query Interview Questions

Q: What is React Query and why is it used?

A: React Query is a data-fetching library that simplifies handling server state in React applications. It provides caching, automatic refetching, and synchronization out of the box.

Q: Difference between `useQuery` and `useMutation`?

A: `useQuery` is for fetching and caching data, while `useMutation` is for creating/updating/deleting data and does not run automatically.

Q: What is the purpose of `queryKey` in React Query?

A: The `queryKey` uniquely identifies a query. React Query uses it for caching, refetching, and avoiding duplicate requests.

Q: How does React Query handle caching?

A: React Query caches query results based on their `queryKey`. Cached data can be reused, avoiding unnecessary API calls.

Q: What is `staleTime` and `cacheTime` in React Query?

A: `staleTime` defines how long data stays fresh before being refetched. `cacheTime` defines how long inactive queries stay in memory before being garbage-collected.

Q: How do you manually update cached data?

A: You can use `queryClient.setQueryData` to directly update cached queries without refetching.

Q: How does React Query handle pagination?

A: React Query provides `useInfiniteQuery` for infinite scroll or paginated data fetching, allowing you to fetch additional pages with `fetchNextPage`.

